AI-POWERED STUDENT AUTOMATION · PROJECT TECHNICAL DOCUMENTATION

CodeSense

An AI-Powered, Hint-First Intelligent Debugging & Programming Learning Platform

Repository: akshayaasundar02/codesense | **Backend:** FastAPI + Python 3.11

Runtimes: Python 3.11 & OpenJDK 17 | **Intelligence:** Groq Llama-3.3-70b

Database: SQLite | **Automation:** n8n Workflow Engine

---

1. Executive Summary & Problem Formulation

1.1 What is the Actual Problem?

When students and junior programmers begin learning programming (in Python and Java), they frequently encounter runtime exceptions and algorithmic bugs:

- `IndexError`: Attempting to access an index beyond the boundaries of an array or list.
- `TypeError`: Performing operations on incompatible types (e.g., concatenating an integer and a string).
- `NameError`: Referencing identifiers, functions, or variables before declaration or initialization.
- `ValueError`: Supplying arguments with an invalid value to functions (e.g., `int("abc")`).
- `IndentationError` / `SyntaxError`: Violating lexical nesting or structural language grammar.

Knowing that an error exists is only the starting point. Students also need to understand:

1. **Understanding the Root Cause:** Why did the program fail at this specific step?

2. **Mental Model Divergence:** Where did the programmer's intent diverge from the computer's actual runtime execution?
3. **Prevention:** How can the student internalize the logic pattern to avoid repeating the mistake in future programs?

Consider this canonical Python off-by-one bug:

```
numbers = [10, 20, 30, 40]
for i in range(len(numbers) + 1):
    print(numbers[i])
```

The student expects all four numbers to be printed. However, the loop executes with `i ∈ {0, 1, 2, 3, 4}`. At `i = 4`, it attempts to access `numbers[4]`. Because zero-indexed lists of length 4 only have valid indices `0, 1, 2, 3`, Python halts with an `IndexError: list index out of range`.

1.2 Why Existing Approaches Fail Learners

Approach	Typical Workflow	Critical Educational Flaw
Traditional Compilers & Debugging	Cryptic terminal tracebacks → Manual Web Search → Trial-and-error edits	High barrier to entry; beginners get overwhelmed by traceback jargon and lack guidance on where to start.
Conventional AI Coding Assistants	Student pastes broken code → LLM spits out fully corrected replacement code	**Premature Spoiler Syndrome:** Bypasses the student's cognitive struggle. The code runs, but the student never internalizes the algorithmic reasoning.
CodeSense Platform	Identify error → 3-Way Cause Breakdown → Progressive Clues (Tier 1 → 2 → 3) → Interactive State Visualization → Transfer Mastery	**Guided Cognitive Apprenticeship:** Protects student reasoning, explains the divergence gap in plain language, visualizes invisible runtime memory, and proves mastery via isomorphic challenges.

1.3 Competition-Ready Problem Statement

🏆 Official Problem Statement

CodeSense — An AI-Powered, Hint-First Intelligent Debugging and Programming Learning Platform

Beginner programmers often struggle to understand, diagnose, and resolve errors in their code. Although conventional debugging tools identify errors and AI coding assistants can generate corrected programs, learners may remain dependent on ready-made solutions without developing strong debugging and problem-solving skills.

CodeSense aims to bridge this gap by providing an interactive, AI-powered

debugging tutor that explains programming errors in simple language, identifies the underlying cause, and delivers progressive hints instead of immediately revealing the corrected code. Learners are encouraged to predict program behaviour, reason about the bug, attempt their own corrections, and validate their solutions through execution and feedback.

The platform further strengthens learning by generating similar practice problems, adapting hints to the learner's progress, and tracking recurring error patterns. Its goal is to transform debugging from a frustrating trial-and-error activity into a guided, measurable, and engaging learning experience that builds independent programming ability.

2. System Architecture

CodeSense is engineered with a modular, decoupled client-server architecture designed for sub-second execution speed, deterministic sandbox security, and pedagogical guidance.

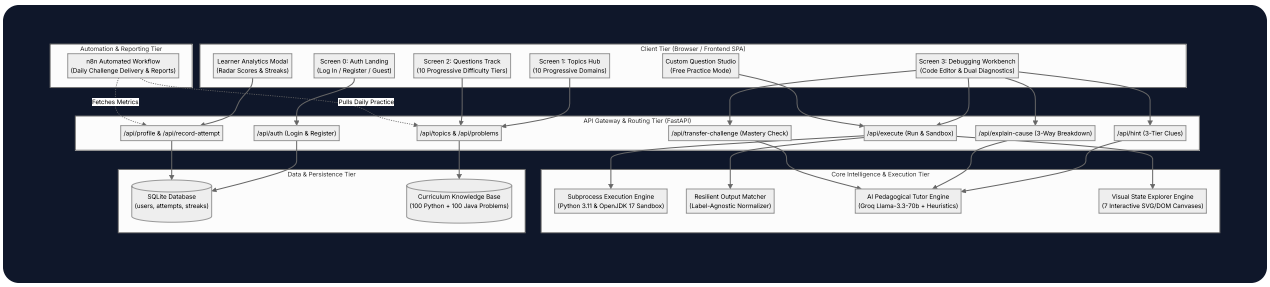


Figure 1: Full System Architecture Diagram of CodeSense

3. The Five Core Innovations

💡 1. Hint-First Debugging Engine

🎨 2. Visual Bug Explorer

Prevents premature solution spoilers. Clues unlock across three distinct pedagogical stages: Line Locator, Conceptual Bridge, and Targeted Direction.

Interactive DOM/SVG representations for 7 runtime failure types: list memory index bounds, type collisions, variable scoping, and loop steps.

🔍 3. 3-Way Cause Breakdown

Deconstructs the complete cognitive chain: Programmer's Intent, Actual Machine Execution, and the Divergence Gap with a memorable Golden Prevention Rule.

🔥 4. Strict Solve-Only Streak Engine

Streaks never increment on login or failed tests. Increments strictly upon verified algorithmic success on the first solve of that calendar day.

🚀 5. Isomorphic Transfer Challenges

Once a bug is repaired, the AI tutor generates a brand new problem with different variables but identical underlying algorithmic structure to prove true mastery.

4. Key Implemented Modules & Features

4.1 Screen 0: Dedicated Authentication Portal

A focused entry portal before the curriculum catalog. Features clean tabs for **Log In** and **Create Account**, plus a **"Continue as Guest →"** shortcut. User passwords are encrypted with SHA-256 salted hashes, and daily problem streaks are persisted in SQLite.

4.2 Pure Algorithmic 10-Tier Curriculum

200 hand-crafted algorithmic challenges (100 Python + 100 Java) spanning 10 topics and 10 difficulty tiers:

- 🟢 **Beginner (Levels 1–3):** List bounds, string reversals, palindrome checks, min/max scanning.
- 🟡 **Intermediate (Levels 4–7):** Two sum hash complements, duplicate removal, zero partitioning, array rotation.
- 🔴 **Advanced (Levels 8–10):** Stock profit tracking, Kadane's maximum subarray sum, trapping rain water.

The catalog includes an active difficulty filter (All 10 Levels, Beginner, Intermediate, Advanced) and workbench header controls with an instant `[ 🚀 Advance to Level X ➔ ]` action banner.

4.3 Resilient Label-Agnostic Test Evaluator

Eliminates the frustration of traditional judges that fail correct code due to label differences (e.g. printing `[5, 4, 3, 2, 1]` instead of `Reversed: [5, 4, 3, 2, 1]`). The engine performs label stripping, whitespace normalization, and multi-line inspection to evaluate genuine logic.

4.4 Target Expected vs Actual Output Display

Dual output panels display side-by-side diagnostic feedback:

- **Your Output:** Raw stdout/stderr produced by the student's program execution.
- **Expected Goal:** Canonical target algorithmic result.

4.5 Review Solved Questions Mode

Students can freely reopen and review any previously solved question without resetting their progress, allowing them to experiment with alternative algorithmic solutions.

5. End-to-End User Learning Journey

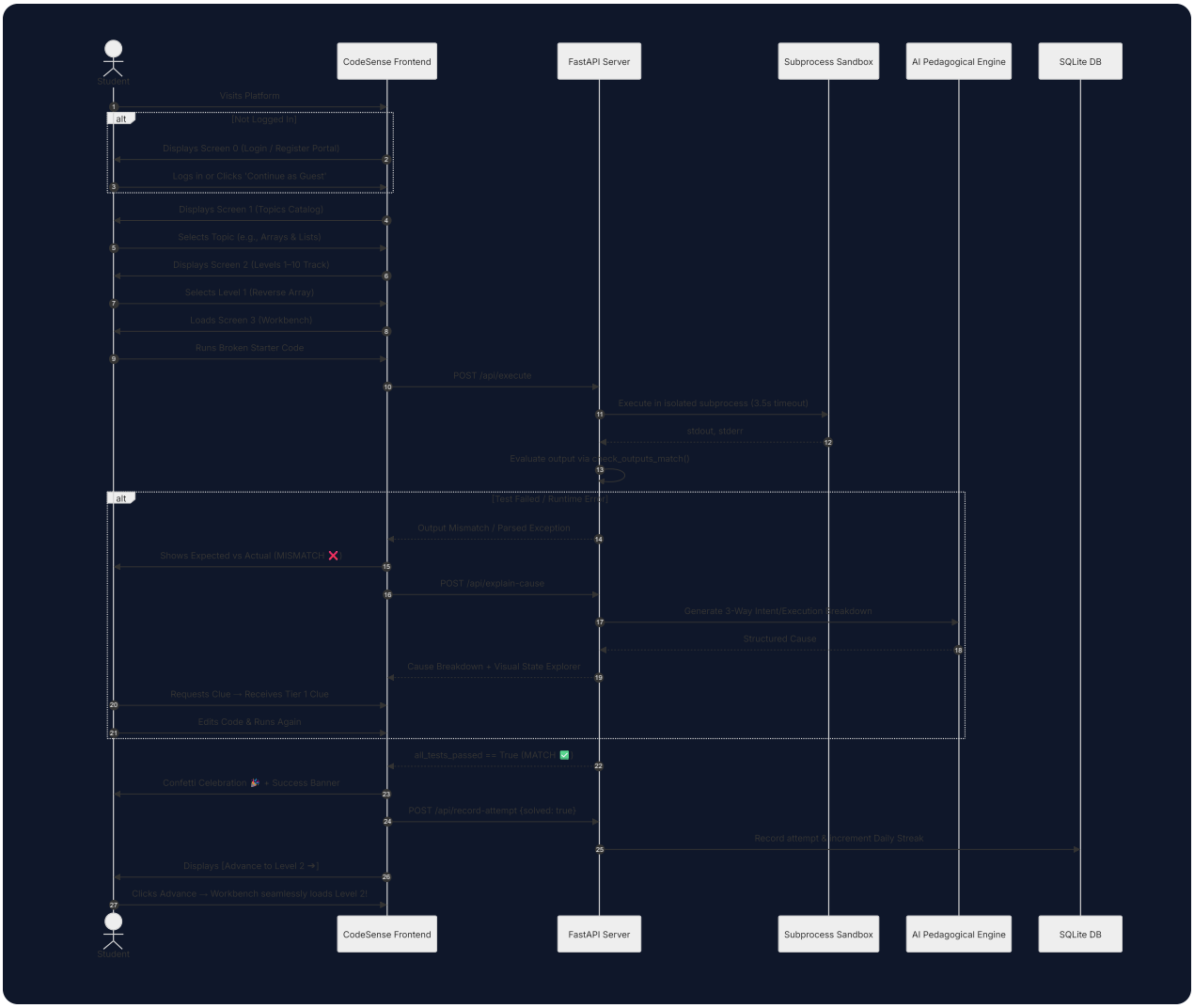


Figure 2: End-to-End User Journey Sequence Diagram

6. Automated Student Workflow (n8n Integration)

Packaged in `codesense_student_automation_workflow.json`, the 9-node automation workflow manages scheduled practice delivery:

- **Node 1: Daily Schedule Trigger** — Fires every morning at 08:00 AM.
- **Node 2: Fetch Student Streak & Profile** — Queries `/api/profile`.
- **Node 3: Retrieve Daily Curated Challenge** — Queries `/api/problems`.
- **Node 4: AI Personalized Clue Generator** — Generates targeted encouragement via Groq LLM.

- **Node 5: Conditional Branching** — Detects at-risk students who missed their streak.
- **Nodes 6 & 7: Morning Challenge Dispatch** — Delivers challenge via Email / Webhook.
- **Nodes 8 & 9: Progress Sync** — Updates student automation metrics.

7. Security & Sandbox Isolation Architecture

To safely execute untrusted student Python and Java programs without risking server integrity, CodeSense implements multi-layered isolation:

- **Subprocess Isolation:** Code runs in standalone Python/Java sub-processes with restricted environment variables, completely stripping API keys.
- **Execution Timeouts:** Hard wall-clock limit of 3.5 seconds terminates infinite loops (`while True`) and resource deadlocks.
- **Memory Safeguards:** Resource limits prevent heap exhaustion and memory attacks.
- **Salted Credential Hashing:** User passwords stored with SHA-256 cryptographic salts.

8. Project Directory Structure

```
codesense/
├── app/
│   ├── config.py              # Application settings, ports, and model parameters
│   ├── curriculum_data.json   # 200 progressive algorithmic challenges (10x10x2)
│   ├── curriculum_data.py     # In-memory accessors and filtering utilities
│   ├── database.py            # SQLite schema, streak calculator, user auth & analy
│   ├── main.py                # FastAPI endpoints, output matching & error handling
│   ├── models.py              # Pydantic validation schemas
│   ├── runner.py              # Sandboxed Python & Java execution engine
│   ├── tutor_engine.py        # 3-tier hints, 3-way cause breakdown & transfer chal
│   └── visualizer_engine.py   # State visualizer schemas (bounds, types, scope, loc
├── static/
│   ├── css/
│   │   ├── style.css          # Glassmorphism design system & responsive styling
│   │   └── visualizer.css     # Memory cell, pointer, and flowchart styling
│   ├── js/
│   │   ├── app.js             # Main SPA coordinator, view routing & auth handlers
│   │   ├── editor.js          # Code editor manager, tab handler & run triggers
│   │   ├── hints.js           # Tiered hints drawer coordinator
│   │   └── visualizer.js      # Dynamic SVG/HTML visual memory renderer
│   └── index.html             # Single-page application root (Screens 0, 1, 2, 3)
├── n8n_workflows/
│   ├── codesense_student_automation_workflow.json # 9-node student automation workf
│   └── AI_STUDENT_AUTOMATION_PROJECT_REPORT.md    # Domain & automation project rep
├── DOCUMENTATION.md           # Markdown comprehensive documentation
├── Dockerfile                 # Debian Bookworm container with OpenJDK 17 + Python
├── requirements.txt           # FastAPI, Uvicorn, Pydantic & Groq dependencies
├── codesense.db               # SQLite persistent database
└── run.py                     # Local development launcher
```